

Education

Nanjing University
2023.09 - 2026.06 (expected)
Master's Degree in *Computer Science and Technology* | PASCAL Lab. Advisor:  Li | Research focus on **PL** and **Program Analysis**.
Teaching Assistant: **Principles and Techniques of Compilers** (Spring 2024)

Beihang University
2019.09 - 2023.06
Bachelor's Degree in *Computer Science and Technology* | GPA 3.84/4.00; awarded **National Scholarship**, **Outstanding Graduate**, and First Prize in the 2021 NSCSCC Compilation System Design Competition.
Teaching Assistant: **Programming in Practice** (Fall 2020), **Object-oriented Design and Construction** (Fall 2021, Spring 2022)

Work Experience

Tencent
2026.05 - Present
Security Department III
Security Technology (Qingyun Program)

NVIDIA
2025.02 - 2025.10
OCG (Optimizing Code Generator) team
GPU Compiler LLVM Backend Intern

- Led the integration of the memory instruction vectorizer between the NVIDIA GPU graphics compiler and NVVM to align with LLVM upstream and reduce maintenance overhead:
 - Designed an encoding scheme for **multi-address graphics memory instructions** to reuse LLVM's memory-access vectorization pipeline;
 - Added alias analysis support for graphics instructions, including detailed analysis of BasicAA and SCEVAA to support new intrinsics;
 - Improved LLVM's alignment inference by solving GCDs of SCEV expressions to infer alignment for loop induction variables;
 - Implemented **more than 10** vectorization optimizations for graphics memory instructions, including **irregular access sequence padding**, improving vectorization success rate by **40%**;
- Contributed to graphics compiler maintenance, fixing multiple bugs in Vectorization and SCEV optimizations;

Rust Foundation Fellowship Program
2024.09 - 2025.09
Rust Foundation Fellowship (approximately 20 fellows globally)
Project Fellow

- Member of the  **Rust-lang Organization** (rust-analyzer-contributors-team) and a **maintainer** of the official Rust IDE rust-lang/rust-analyzer, ranked in the **top 1%** of contributors; handled issues and PR reviews, and helped maintain other Rust community projects such as rust-clippy;
- Delivered features such as control-flow highlighting and snapshot-test updates, and resolved numerous bugs to improve code understanding and code generation;
- Implemented a **SIMD** NEON version of the Unicode line-breaking module, achieving a **6.5x** speedup on ARM;
- Incident response for v0.3.1992**: Identified a dependency-graph search flaw within **3 hours** and designed a new algorithm to resolve a critical resource-exhaustion bug discovered four hours after release, containing the impact on the community.

Projects

VIDE
VIDE
Modern IDE for Chip Frontend Design · Master's Thesis Project
Rust / SystemVerilog

- Implemented a **semantic analysis framework** and IDE infrastructure for SystemVerilog, enabling modern IDE capabilities for chip design;
- Built on an **incremental computation** architecture, designed and implemented an incremental analysis IR and incremental analysis passes, allowing the analyzer to produce accurate results without full recomputation;
- Project reaches **industry-leading standards** in functionality and performance: completed **more than 10** modern IDE features for SystemVerilog, including code navigation, semantic refactoring, and code completion, with **millisecond-level** latency through incremental semantic analysis; based on the Language Server Protocol, compatible with mainstream editors such as VS Code and Emacs;

Ailurus

Rust

Programming Language and Toolchain Design Exploration · Personal Interest Project

- Based on **Martin-Löf type theory**; supports **dependent types**, dependent pattern matching, and inductive datatypes. Implements propositional equality and uses Normalization by Evaluation for equivalence checking, enabling simple theorem proving;
- Uses **typeclass-based** ad-hoc polymorphism and implements **operator overloading** on top of it; implements a **module system** for namespace management and encapsulation;

- Serves as an experimental platform for exploring collaborative design architectures for modern programming language toolchains, such as compilers and IDEs, improving development efficiency and maintainability.

🔗 Open Source Contributions

- 🐙 Maintain the official Rust IDE (language server) [🐙 rust-lang/rust-analyzer](#) also contributed to Rust community projects including [🐙 rust-lang/rust](#), [🐙 rust-lang/rust-clippy](#), [🐙 rust-lang/rustup](#), [🐙 rust-lang/rust-mode](#)
- [🐙 llvm/llvm-project](#), [🐙 clangd/vscode-clangd](#), [🐙 LuaLS/lua-language-server](#), [🐙 google/autocxx](#), [🐙 yuin/goldmark](#), [🐙 moonbitlang/tree-sitter-moonbit](#), [see more projects on GitHub](#).

📋 Skills

Programming Languages	Language-agnostic capabilities. Proficient in C, C++, Rust, Java, Python, JavaScript/TypeScript, Verilog/SystemVerilog, and EmacsLisp; studied and used Ruby, Swift, OCaml, Haskell, Coq, Agda, etc.;
Programming Language Theory	• Fundamentals including formal semantics, type theory, computation models, and automata; experience with proof assistants such as Coq and Agda; • (Type Systems) Theory and implementation of Hindley-Milner, subtyping, System F, dependent types, etc.; • (Static Analysis) Data-flow analysis, control-flow analysis, IFDS, and pointer analysis with varying sensitivities.
Compiler Design	3 years of experience. Proficient in full-pipeline compiler development from parsing to code generation, and familiar with multiple IRs (ANF, SSA, CPS, etc.): • (Language Implementation) Compilation for object-oriented, functional, and other paradigms, and implementation of language features such as bidirectional type checking; • (IDE Development) 2 years of experience with IDE architecture based on incremental computation and LSP, plus editor plugin development; familiar with rust-analyzer and clangd; • (Compiler Optimization) Middle-end and backend analyses and optimizations, including Mem2Reg, GVN, RegAlloc, List Scheduling, etc.; familiar with LLVM analysis and optimization implementations, the LLVM codebase, LLVM-IR, and MLIR;
Application Development	Ruby on Rails, Django, SwiftUI, and other development frameworks; PostgreSQL, Redis, and other databases; Docker, CI/CD configuration, and other DevOps work;
Development Environment	Proficient in Emacs / VS Code, comfortable working on macOS / Linux, and skilled at using generative AI tools to improve efficiency.

📁 Misc

- **Talks:** *Exploring rust-analyzer: from Incremental Computation to Modern IDE* (RustChinaConf 2025)